webSPDZ: Performance, Security and Reachability Metrics

An analysis of the benchmark data and of the ATLAS browser demo

Author

October 2026

Abstract

webSPDZ compiles the MP-SPDZ multi-party computation (MPC) engine to WebAssembly, so that every party of a secure computation runs in an ordinary browser tab and the parties talk peer-to-peer over WebRTC. This report collects quantitative evidence about the project from the benchmark data in `Paper-Benchmarks/Data` (79 measurement files, four engines, four protocols, 3 to 13 parties, 10^2 to 10^6 elements) and from the 4-party ATLAS "surplus matching" demo built on top of it. The main findings are: webSPDZ runs a secure dot product of 10^5 elements among 3 parties in **1.16 s**, which is **229× faster than JIFF** and **5.7× faster than MPyC-Web**, the two other web MPC engines measured. Its slowdown compared with native MP-SPDZ is **2–18×** and shrinks with larger inputs and with more parties. The engine needs no installation, a 4.6–7.2 MB download, and, after the changes described in Section 6, runs across several devices on one local network.

1 Scope and methodology

Benchmark. All engines evaluate the same task: the secure inner product of two secret-shared integer vectors of length 10^k, split into an *input* phase (secret-sharing the vectors) and an *inner-product* phase (the actual computation). The *full* time covers both phases.

Engines and environment. The data files record the following set-up (`Paper-Benchmarks/MPC-Engines/MPC-engines_details_version-etc.md` and the `_meta` field of each file):

- **webSPDZ**: MP-SPDZ source of 20 June 2024 (commit `18e934f`) compiled to 64-bit WebAssembly; Firefox Nightly 134.0a1.
- **MP-SPDZ** (native): the same source, versions 0.3.9 and 0.3.7.
- **JIFF**: JavaScript MPC library (June 2024, commit `f5a22d8`), run on Node.js, which uses the same V8 engine as Chrome.
- **MPyC-Web**: Python MPC compiled to WebAssembly (February 2024, commit `5b72f19`); Firefox 132.

Every party ran in a Docker container limited to 4 CPUs and 4 GB of RAM, in a LAN setting. Each configuration was run 3 times. Measurements were taken between October and November 2024.

Metric. A secure computation finishes only when its slowest party finishes. For each configuration we therefore report the *maximum over the parties* of the 3-run average. Slowdowns and speed-ups are ratios of these values.

Caveats. (i) Timers have 1 ms resolution, so native MP-SPDZ results at 10^2 and 10^3 elements (0–1 ms) are not used for ratios. (ii) JIFF with 6 parties is a single run (`Data/JIFF/note.md` reports that this run was unstable). (iii) The MPyC-Web file contains malformed JSON in its per-run section, so only its averages are used. (iv) The dot-product benchmark measures the engine, not the ATLAS demo application, which has no timing data yet (Section 4).

2 Performance and speed

2.1 Runtime as a function of input size

Table 1 and Figure 1 show the full runtime for growing vectors. Three observations stand out.

Protocol (parties)	Engine	10^2	10^3	10^4	10^5	10^6
Replicated-Ring (3)	webSPDZ	0.011	0.016	0.037	0.258	1.375
	MP-SPDZ	<0.001	0.001	0.007	0.072	0.683
Shamir passive (3)	webSPDZ	0.019	0.022	0.132	1.159	6.014
	MP-SPDZ	<0.001	0.001	0.012	0.116	1.148
	JIFF	0.371	2.413	23.894	265.105	–
	MPyC-Web	–	–	–	6.577	–
Rep4-Ring (4)	webSPDZ	0.046	0.052	0.218	1.833	16.563
	MP-SPDZ	0.001	0.001	0.012	0.119	1.150

Table 1: Full runtime in seconds of a secure dot product with n elements (slowest party, mean of 3 runs).

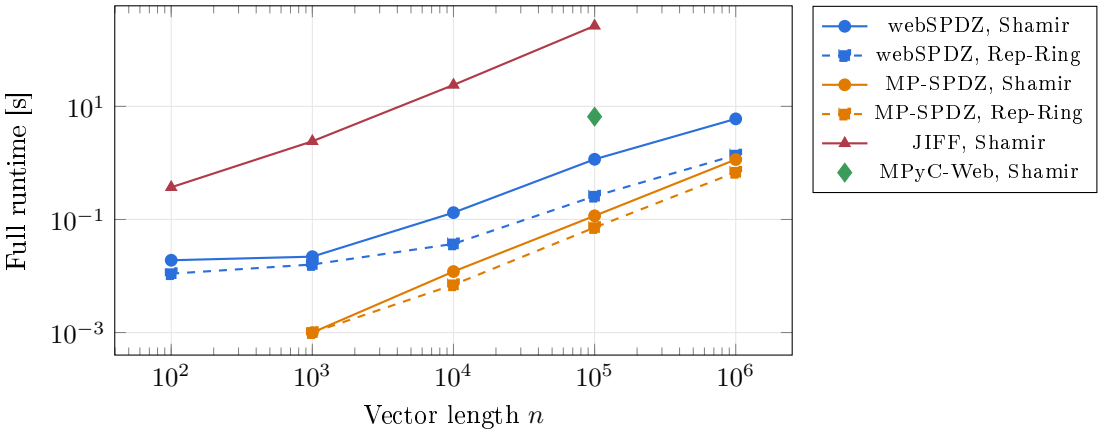


Figure 1: Runtime against input size, 3 parties, log–log scale. The web engines are drawn in blue (webSPDZ), red (JIFF) and green (MPyC-Web), the native baseline in orange.

1. **webSPDZ is two orders of magnitude faster than JIFF.** With Shamir's protocol and 3 parties, the speed-up grows from 19.5× at 10^2 elements to 110× at 10^3, 181× at 10^4 and **229× at** 10^5 (1.16 s against 265 s). JIFF's time is dominated by the inner-product phase (245 s of 265 s), which webSPDZ completes in 0.11 s. Compared with MPyC-Web, the other WebAssembly engine, webSPDZ is **5.7**× faster at 10^5 elements.

2. **There is a fixed start-up cost of roughly 10–50 ms.** Below 10^3 elements, webSPDZ runtimes barely change (11–16 ms for Replicated-Ring, 46–52 ms for Rep4-Ring). This is the cost of the browser runtime and the communication rounds, not of the data.

3. **Runtime then grows linearly.** From 10^4 to 10^6 elements every engine follows a slope close to 1 on the log–log plot, as expected for a computation whose communication is proportional to n.

2.2 Cost of running in the browser

Table 2 compares webSPDZ with native MP-SPDZ built from the same source. The browser costs **2.0× to 18.2×**, depending on the protocol. The overhead shrinks as inputs grow, because the fixed browser costs are amortized: for Replicated-Ring it falls from 5.3× to 2.0×, for Shamir from 11.0× to 5.2×. The actively secure Rep4-Ring protocol has the highest overhead (14–18×). It sends more messages and checks them, and each message likely pays the extra cost of WebRTC and the browser's event loop.

Protocol	10^4	10^5	10^6
Replicated-Ring, 3 parties (passive)	5.3×	3.6×	**2.0**×
Shamir, 3 parties (passive)	11.0×	10.0×	5.2×
Rep4-Ring, 4 parties (active)	18.2×	15.4×	14.4×

Table 2: Slowdown of webSPDZ compared with native MP-SPDZ 0.3.9.

Throughput. At 10^6 elements webSPDZ processes about **727 000 elements/s** with Replicated-Ring, 166 000 elements/s with Shamir and 60 000 elements/s with Rep4-Ring (native Replicated-Ring: 1.46 million elements/s).

Where the time goes. The input phase takes **87–96%** of the full runtime in webSPDZ at 10^6 elements (87% Replicated-Ring, 91% Shamir, 96% Rep4-Ring). Secret-sharing the inputs is pure communication, so the bottleneck of browser MPC is the network channel, not the WebAssembly computation. The computation itself (0.18–0.69 s for 10^6 products) is close to native speed.

Stability. The standard deviation between runs stays below about 8% of the mean for webSPDZ (worst case: Shamir with 5 parties, 0.105 s on 1.42 s). As a reproducibility check, native MP-SPDZ 0.3.7 and 0.3.9 differ by less than 5% on every Shamir configuration.

2.3 Scalability with the number of parties

Figure 2 varies the number of parties from 3 to 13 for 10^5 elements with Shamir's protocol.

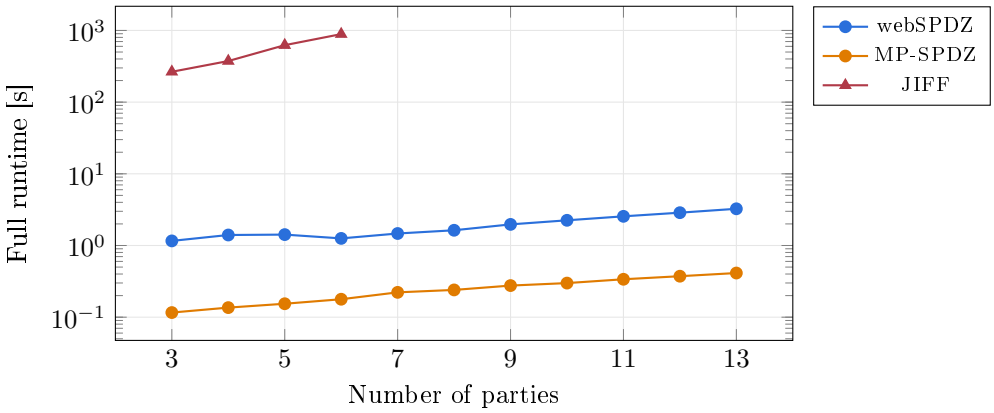


Figure 2: Runtime against the number of parties, Shamir passive, 10^5 elements.

- webSPDZ goes from 1.16 s with 3 parties to **3.25 s with 13 parties** (2.8× for 4.3× more parties). Native MP-SPDZ grows by 3.6× over the same range, so the browser overhead *decreases* with more parties: from 10.0× at 3 parties to 7.9× at 13.
- From 6 to 13 parties, every additional party adds about 0.29 s to webSPDZ (0.034 s natively).
- JIFF triples its runtime from 3 to 6 parties (265 s to 889 s), and at 6 parties webSPDZ is **about 700× faster**. JIFF was not measured beyond 6 parties.

3 Security

3.1 Supported security models

webSPDZ inherits MP-SPDZ's protocols. Table 3 lists those compiled for the browser in this repository. They cover both honest- and dishonest-majority settings and both passive (semi-honest) and active (malicious) adversaries.

Protocol (binary)	Corruption tolerated	Adversary	Download
Replicated-Ring (`replicated-ring-party`)	1 of 3	passive	6.6 MB
Rep4-Ring (`rep4-ring-party`)	1 of 4	active	6.7 MB
Shamir (`shamir-party`)	$t < n/2$	passive	7.2 MB
ATLAS (`atlas-party`)	$t < n/2$	passive	4.6 MB
Semi2k (`semi2k-party`)	$n - 1$ of n	passive	7.1 MB

Table 3: Protocols available as browser builds. "Download" is the size of the uncompressed `.wasm` module.

3.2 Security of the deployment

- **Inputs never leave the browser in clear.** Each party secret-shares its input locally. Only shares travel between the parties.
- **Peer-to-peer channels are encrypted.** WebRTC data channels always use DTLS, so the protocol messages between parties are encrypted in transit.
- **The servers do not see inputs or protocol messages.** The signaling server only relays the WebRTC handshake (offers, answers, ICE candidates) and the lobby state (seats and "ready" flags). It is not on the data path once the peers are connected. The web server only delivers static files.
- **Secure contexts.** Both servers use TLS (`https://`, `wss://`), and the web server sends the COOP/COEP headers required for the WebAssembly threads.
- **Parameters.** The demo program is compiled with 64-bit integers and a statistical security parameter of 40 bits: what secret comparisons leak is bounded by a statistical distance of 2^{-40}.

Limitations. (i) ATLAS and Shamir are semi-honest: a party that deviates from the protocol can corrupt the result without being detected. The active Rep4-Ring protocol is available, but in the browser it is about 7× slower than the passive Replicated-Ring at 10^5 elements (1.83 s against 0.26 s; 4 against 3 parties, so not a strict like-for-like comparison). (ii) The repository uses a self-signed certificate, which the users accept manually. (iii) Parties must trust the web server to deliver the correct code; a malicious server could serve a modified `.wasm`. This trust assumption is common to all web-based MPC.

4 The ATLAS surplus-matching demo

The application in `Programs/Source/surplus_matching_private.mpc` lets 4 parties, each with 4 private resource levels, find the best pairing in which surpluses cover deficits. Each party learns only its own partner. It runs under ATLAS with $n = 4$ and $t = 1$. Compiling it reports the following upper bounds on its cost:

Metric	Value
Private inputs per party	4 (compiler bound: 5)
Secret multiplications (triples)	23 892
Random secret bits (for comparisons)	21 201
Values opened	206
Communication rounds (VM rounds)	38
WebAssembly module	4.6 MB

Table 4: Cost of the surplus-matching program, as reported by the MP-SPDZ compiler (`compile.py`).

The program is small compared with the benchmarks above: about 24 000 multiplications against 10^5–10^6 in the dot products. With only 38 communication rounds, its runtime on a LAN should be dominated by start-up (downloading the 4.6 MB module and connecting the 4 peers), not by the computation. This is an estimate: the demo has not been timed yet. Timing it on 4 separate devices is the natural next measurement.

5 Reachability

No installation. Participants only need a modern browser. There is no native binary, compiler or library to install, and nothing depends on the operating system or CPU architecture. The whole deployment is two small servers, started with one command (`docker compose up -build`).

Browser requirements. The build uses 64-bit WebAssembly (Memory64), WebAssembly threads (shared array buffers), native WebAssembly exceptions and 500 MB of initial memory. Table 5 summarizes what this means for each kind of device.

Network reachability.

- **Same machine** (several tabs): supported from the start.
- **Several devices on one local network**: supported after the changes in Section 6. Wi-Fi networks with client isolation, common on campus and guest networks, block it.

Device	Status	Reason
Laptop/desktop, Chrome ≥ 133 or Firefox ≥ 134	supported	Memory64 available
Android, current Chrome	likely	Memory64 available; 500 MB may fail on low-RAM phones
iPhone/iPad (any browser)	unlikely	WebKit lacked Memory64 at the time of writing

Table 5: Device support for the browser parties.

- **Different networks**: possible through an overlay VPN (Tailscale, ZeroTier) without code changes. Without a VPN, peers behind strict NATs need a TURN relay, which is not configured yet (only a public STUN server is).

6 Innovations

Of webSPDZ as an engine.

- A **general-purpose** MPC engine in the browser: the full MP-SPDZ virtual machine, with its protocols and its high-level programming language, rather than a single hand-written protocol.
- **Peer-to-peer** communication between browsers over WebRTC, with a server used only to introduce the parties.
- Engineering of a large C++ code base for the web: 64-bit WebAssembly, cryptographic libraries (libsodium, OpenSSL) ported to WebAssembly, and the networking moved off the browser's UI thread.

Of this project's ATLAS demo.

- **ATLAS in the browser**, which required pinning Emscripten 4.0.6 and enabling native WebAssembly exceptions.
- **Private inputs typed live in the page.** Inputs are written into the module's virtual file system at run time. Before, every party's input was compiled into the shared `.wasm` bundle, so every participant downloaded everyone's inputs. This was the first limitation listed in the application paper, and it is now removed.
- **One shared link instead of hand-written URLs.** A lobby assigns seats automatically and starts the computation once all parties are ready. The lobby only carries seat numbers and ready flags, never input values.
- **Multi-device operation on a LAN.** The lobby now passes the signaling server's address (`-ss`) to each party. Before, every party looked for the signaling server on its own `localhost`, which worked only on the server machine.
- **Reproducible deployment** with a Docker image that bundles both servers.

7 Conclusions

The benchmark data show that general-purpose MPC in the browser is practical. webSPDZ is 2–15× slower than native MP-SPDZ for large inputs, but two orders of magnitude faster than the JavaScript engine JIFF and about 6× faster than MPyC-Web. Its overhead shrinks with larger inputs and with more parties, and its cost is dominated by communication, not by WebAssembly. The project combines this with a choice of security models, an encrypted peer-to-peer data path, no installation, and now a usable multi-device workflow. The main open points are measuring the ATLAS demo on separate devices, adding a TURN relay for networks other than a LAN, reaching iOS, and moving the demo to an actively secure protocol.

Data and reproducibility. All numbers in Sections 2 and 3 come from the JSON files in `Paper-Benchmarks/Data`; the ATLAS figures in Section 4 come from compiling `surplus_matching_private.mpc`. The webSPDZ design is described in Buchsteiner et al., *webSPDZ: Versatile MPC on the Web*, Cryptology ePrint Archive 2025/487.